

EDUHUB

Backend Technical Documentation

Multi-Tenant Hierarchical Educational Management System

Document Version 1.0

Technology Stack — Node.js (ES Modules), Express.js, MongoDB / Mongoose, JWT Authentication

Prepared for: Idrees, Habil — Backend Development Team

By Muhammad Junaid

Table of Contents

1. Project Overview	3
2. Objectives and Scope.....	3
3. System Requirements.....	3
4. User Roles and Responsibilities	4
5. Functional Requirements	5
6. Non-Functional Requirements	5
7. System Architecture	6
8. Backend Architecture	7
9. Module / Component Breakdown.....	8
10. API Design.....	8
11. Authentication & Authorization	10
12. Database Design	11
13. Entity Relationships	17
14. Main Business Workflows.....	18
15. Request / Response Flow	19
16. Error Handling.....	19
17. Validation.....	19
18. Security Considerations.....	20
19. Logging and Monitoring.....	20
20. Deployment Architecture	20
21. Development Workflow & Work Distribution.....	21
22. Testing Strategy	23
23. Future Scalability Considerations	23
24. Conclusion	24

1. Project Overview

EduHub is a backend system for managing educational institutions through a strict three-tier administrative hierarchy: Super Admin, Institute Admin, and Campus Manager, with Teachers and Students operating within a specific campus. The system supports institute and campus registration, staff appointment, admissions, timetabling, examination scheduling, attendance tracking, fee management, and academic performance recording.

Each entity in the system is scoped to its owning institute and/or campus to prevent cross-tenant data access, making EduHub a multi-tenant system at the data-access level rather than at the infrastructure level: a single shared database, logically partitioned by `instituteId` and `campusId`.

2. Objectives and Scope

Objectives

- Provide a single backend that supports four distinct operating contexts (Super Admin, Institute Admin, Campus Manager, and the campus-level academic lifecycle) without data leakage between tenants.
- Provide a clear contract between two developers (Idrees and Habil) so both tracks can be built and tested independently before integration.
- Provide a RESTful API consumed by a separate frontend, which is out of scope of this document.

In Scope

- Authentication and role-based authorization.
- Institute and campus registration and administration.
- Faculty and student directory management.
- Class and exam scheduling.
- Attendance tracking for faculty and students.
- Fee voucher issuance and payment status tracking.
- Academic performance — marks, grades, GPA — recording.

Out of Scope

- Frontend implementation.
- Payment gateway integration — fee status is recorded manually; no live payment processing is specified.
- Notifications by email or SMS — not specified in the requirements.
- Mobile applications.

Note. Any capability not explicitly listed in the source requirements — for example notifications, payment gateway integration, or file uploads beyond image URLs — is treated as out of scope for this version.

3. System Requirements

Category	Requirement
Runtime	Node.js, ES Modules (import / export syntax, not CommonJS)

Category	Requirement
Web framework	Express.js
Database	MongoDB, accessed via the Mongoose ODM
Authentication	JSON Web Tokens (JWT), bearer header
Password storage	Salted hash via bcryptjs; never stored in plaintext
API style	REST, versioned under /api/v1
Testing tool	Postman — manual endpoint verification, per requirements

4. User Roles and Responsibilities

Role	Scope	Core Responsibilities
Super Admin	Global (platform-wide)	Registers institutes, appoints Institute Admins, views global metrics, activates or deactivates any user
Institute Admin	One institute	Registers campus branches, appoints Campus Managers, monitors cross-branch statistics for the institute
Campus Manager	One campus	Manages faculty directory, student admissions, timetables, exam datesheets, daily attendance, fee vouchers, and academic grades
Teacher	One campus (assigned)	Subject of scheduling, attendance, and performance records
Student	One campus (enrolled)	Subject of enrollment, attendance, fee, and performance records; may self-register via the public register endpoint

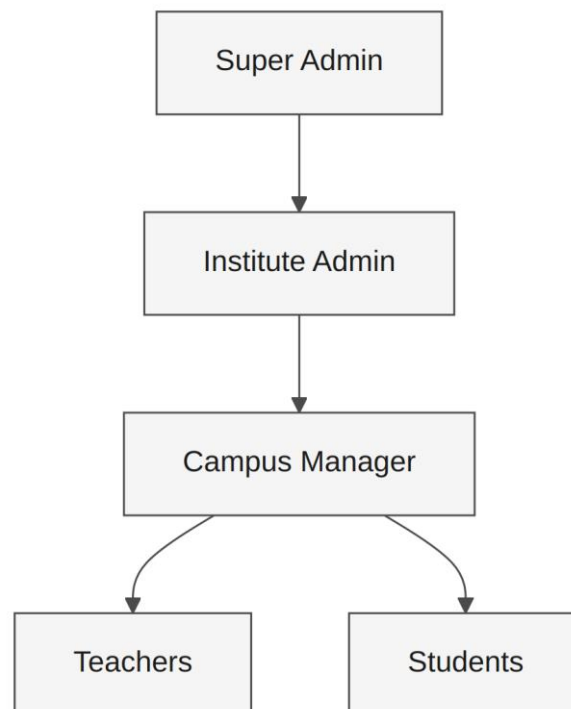


Figure 1 — Role hierarchy

Note. The requirements do not define dedicated Teacher- or Student-facing API endpoints, such as a teacher viewing their own timetable or a student viewing their own grades. Only Campus Manager-side CRUD endpoints over teacher and student data are specified. This gap is carried forward into Section 23.

5. Functional Requirements

ID	Requirement	Owning Role
FR-1	Users authenticate via email and password and receive a JWT	All roles
FR-2	A new user can self-register through a public endpoint	Public / Student
FR-3	Super Admin can register institutes, optionally with an inline Institute Admin	Super Admin
FR-4	Super Admin can create, update and delete institutes and campuses globally	Super Admin
FR-5	Super Admin can activate or deactivate any user account	Super Admin
FR-6	Institute Admin can create, update and delete campus branches under their institute	Institute Admin
FR-7	Institute Admin can appoint or reassign a Campus Manager to a branch	Institute Admin
FR-8	Institute Admin can view institute-wide KPIs: campuses, managers, faculty, students, fee collection	Institute Admin
FR-9	Campus Manager can manage the faculty directory, which also creates the underlying User	Campus Manager
FR-10	Campus Manager can manage student admissions, which also creates the underlying User	Campus Manager
FR-11	Campus Manager can create and manage class timetables	Campus Manager
FR-12	Campus Manager can create and manage exam datesheets	Campus Manager
FR-13	Campus Manager can record daily faculty attendance	Campus Manager
FR-14	Campus Manager can record subject-wise student attendance	Campus Manager
FR-15	Campus Manager can issue fee vouchers and update payment status	Campus Manager
FR-16	Campus Manager can record and update student marks, grades and GPA	Campus Manager
FR-17	Any authenticated user can view and update their own profile	All roles

6. Non-Functional Requirements

Category	Requirement
Data isolation	No user may read or write data belonging to another institute or campus; enforced at the middleware layer
Auditability	All core records from User downward carry creation and update timestamps
Consistency	Uniqueness constraints on natural keys (email, rollNo per campus, voucherNo) and compound uniqueness on attendance records prevent duplicate entries

Category	Requirement
Maintainability	Codebase is split into two independently developed, loosely coupled tracks joined only through IDs and shared middleware
Availability	Standard single-region deployment is assumed; no explicit SLA is defined in the requirements

Note. Performance targets, uptime SLAs, and concurrency limits are not specified in the source requirements. Sections 18–20 propose reasonable defaults and label them as proposed, not mandated.

7. System Architecture

The system follows a conventional three-layer backend: an HTTP layer (routes and middleware), a business logic layer (services), and a persistence layer (Mongoose models over MongoDB). The frontend is a separate, out-of-scope client consuming the versioned REST API.

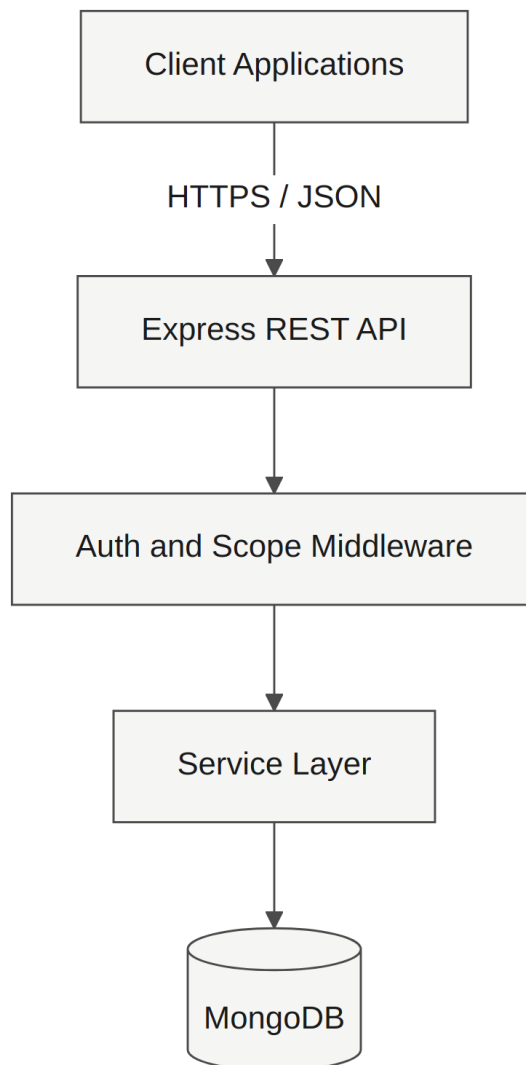


Figure 2 — High-level system architecture

8. Backend Architecture

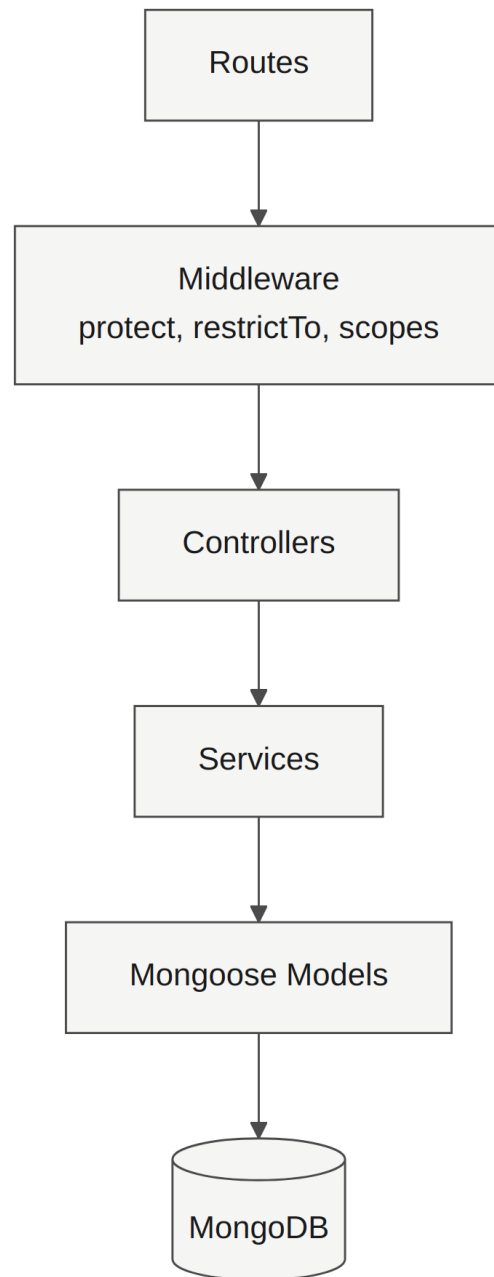


Figure 3 — Layered backend architecture

- Routes — Define the URL surface and attach the relevant middleware chain per endpoint (Section 10).
- Middleware — Performs authentication (protect), authorization (restrictTo), and tenant scoping (instituteAdminScope, campusAdminScope).
- Controllers — Parse the HTTP request and response and delegate business logic to services.
- Services — Contain all business rules — for example, atomically creating a User and a TeacherProfile — and are the only layer that talks to Mongoose models.
- Models — Define schema, validation, and relationships.

This layering keeps HTTP concerns in the controllers separate from business rules in the services, which is what allows Idrees and Habil to build their tracks independently against a shared contract of models and middleware.

9. Module / Component Breakdown

Each module below is mapped to its owning developer. Full day-by-day execution plans and the integration protocol between the two tracks are in Section 21.

Module	Models	Services / Controllers	Routes	Owner
Auth & Identity	User	authService, authController	authRoutes.js	Idrees
Platform Governance	Institute, Campus	superAdminService, superAdminController	superAdminRoutes. js	Idrees
Institute Administration	Campus (scoped)	instituteAdminService, instituteAdminController	instituteAdminRoutes.js	Idrees
Campus Operations	TeacherProfile, StudentProfile, ClassSchedule, ExamSchedule	campusAdminService, campusAdminController	campusAdminRoutes.js	Habil
Attendance	TeacherAttendance, StudentAttendance	campusAdminService (attendance functions)	campusAdminRoutes.js	Habil
Finance	FeeRecord	campusAdminService (fee functions)	campusAdminRoutes.js	Habil
Academics	Performance	campusAdminService (performance functions)	campusAdminRoutes.js	Habil

10. API Design

All endpoints are versioned under `/api/v1`. The tables below consolidate the requirements' Route Security Matrix, grouped by module.

Auth

Method	Endpoint	Access	Middleware
POST	<code>/api/v1/auth/login</code>	Public	—
POST	<code>/api/v1/auth/register</code>	Public	—
GET	<code>/api/v1/auth/me</code>	Authenticated	protect
PUT	<code>/api/v1/auth/profile</code>	Authenticated	protect

Super Admin

All routes require `protect` and `restrictTo('super_admin')`.

Method	Endpoint	Purpose
GET	/api/v1/super-admin/stats	Global platform analytics
GET / POST	/api/v1/super-admin/institutes	List / register institutes
GET / PUT / DELETE	/api/v1/super-admin/institutes/:id	Manage a single institute
POST	/api/v1/super-admin/institutes/:id/assign-admin	Assign Institute Admin
GET / POST	/api/v1/super-admin/institute-admins	List / create Institute Admins
GET / POST	/api/v1/super-admin/campuses	Global campus list / creation
PUT / DELETE	/api/v1/super-admin/campuses/:id	Update / delete a campus
GET	/api/v1/super-admin/users	Global user list
PATCH	/api/v1/super-admin/users/:id/toggle-status	Activate / deactivate account

Institute Admin

All routes require protect and instituteAdminScope.

Method	Endpoint	Purpose
GET	/api/v1/institute-admin/stats	Institute-wide KPIs
GET	/api/v1/institute-admin/profile	Assigned institute details
GET / POST	/api/v1/institute-admin/campuses	List / create branches
GET / PUT / DELETE	/api/v1/institute-admin/campuses/:id	Manage a branch
POST	/api/v1/institute-admin/campuses/:id/assign-manager	Appoint / reassign Campus Manager
GET / POST	/api/v1/institute-admin/managers	List / appoint Campus Managers

Campus Manager

All routes require protect and campusAdminScope.

Method	Endpoint	Purpose
GET	/api/v1/campus/overview	Campus KPI dashboard
GET / POST	/api/v1/campus/teachers	Faculty directory / hiring
GET / PUT / DELETE	/api/v1/campus/teachers/:id	Manage a faculty record
GET / POST	/api/v1/campus/students	Student directory / admissions
GET / PUT / DELETE	/api/v1/campus/students/:id	Manage a student record
GET / POST	/api/v1/campus/schedules	View / create timetable entries
PUT / DELETE	/api/v1/campus/schedules/:id	Update / remove a class
GET / POST	/api/v1/campus/exams	View / schedule exams
PUT / DELETE	/api/v1/campus/exams/:id	Update / delete an exam record
GET / POST	/api/v1/campus/attendance/teachers	Faculty attendance

Method	Endpoint	Purpose
GET / POST	/api/v1/campus/attendance/students	Student attendance
GET / POST	/api/v1/campus/fees	Fee records / voucher issuance
PATCH	/api/v1/campus/fees/:id/status	Mark voucher paid / overdue
GET / POST	/api/v1/campus/performance	View / record grades and GPA

11. Authentication & Authorization

Authentication

- POST /auth/login verifies the password hash and, if isActive, issues a signed token containing the user ID.
- Every protected request supplies the token via the Authorization: Bearer <token> header.
- The protect middleware verifies the token, loads the user, and rejects inactive accounts.

Authorization

Middleware	Purpose
protect	Validates the JWT and attaches the authenticated user to the request
restrictTo(...roles)	Rejects the request unless the user's role is in the allowed list
instituteAdminScope	Resolves the caller's own institute and blocks access to any other institute's data
campusAdminScope	Confirms the caller — Campus Manager or Institute Admin — is authorized over the target campus

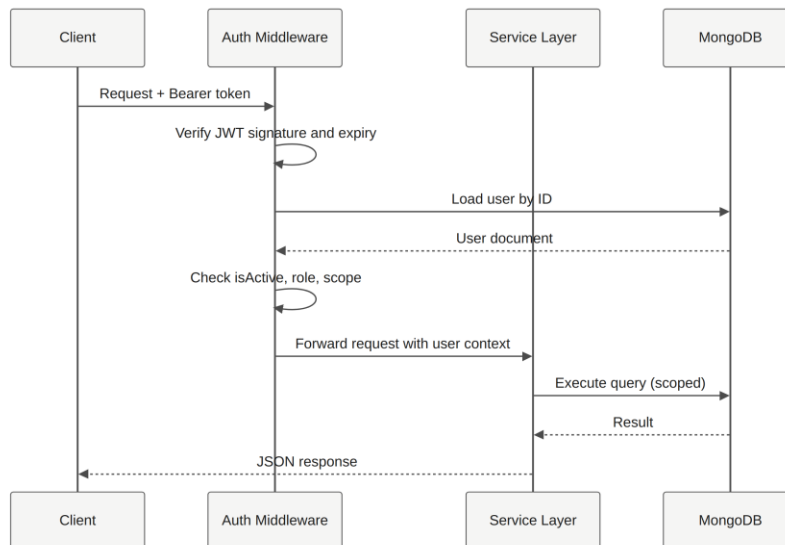


Figure 4 — Authentication and authorization flow

Role-to-scope resolution happens at login: institute_admin accounts are populated with assignedInstitute; campus_manager and campus_admin accounts are populated with assignedCampus, so downstream scoping middleware does not need to re-derive this on every request.

12. Database Design

MongoDB is used with Mongoose schemas. Eleven collections make up the data model. Each is listed below with its owning developer (see Section 21 for the full work division) and full field-level detail, so both developers can implement their schemas directly from this document.

#	Model	Purpose	Owner
1	User	Central authentication account for every role	Idrees
2	Institute	Educational network / organization	Idrees
3	Campus	Physical branch under an institute	Idrees
4	TeacherProfile	Faculty employment and academic metadata	Habil
5	StudentProfile	Student enrollment and academic metadata	Habil
6	ClassSchedule	Weekly timetable entries	Habil
7	ExamSchedule	Examination datesheets	Habil
8	TeacherAttendance	Daily faculty attendance	Habil
9	StudentAttendance	Subject-wise student attendance	Habil
10	FeeRecord	Fee vouchers and payment ledger	Habil
11	Performance	Marks, grades, GPA	Habil

Key Constraints

- User.email — unique, lowercase.
- StudentProfile.rollNo — unique per campus.
- FeeRecord.voucherNo — unique.
- TeacherAttendance — compound unique index on (teacherProfileId, date).
- StudentAttendance — compound unique index on (studentProfileId, subject, date).

Schema Reference — Full Field Definitions

The tables below define every field, type, and constraint per model, so each schema can be implemented directly without cross-referencing another document.

Model 1: User — Owner: Idrees

Central authentication account for every person interacting with the platform.

Field	Type / Format	Constraints
name	string	Required, trimmed
email	string	Required, unique, lowercase, trimmed — primary login identifier
passwordHash	string	Required, hidden by default from queries
role	enum	super_admin, institute_admin, campus_admin, campus_manager, teacher, student

Field	Type / Format	Constraints
instituteId	ObjectId → Institute	Optional for Super Admin; mandatory for Institute Admin; populated for campus staff/students
campusId	ObjectId → Campus	Optional for Super Admin & Institute Admin; mandatory for Campus Manager, Teachers, Students
phone	string	Trimmed
avatar	string	Profile photo URL
isActive	boolean	Defaults to true
timestamps	Date (createdAt/updatedAt)	Automatic

Relationships:

- Belongs to one Institute (optional).
- Belongs to one Campus (optional).
- One-to-one with TeacherProfile (if role is teacher) or StudentProfile (if role is student).

Model 2: Institute — Owner: Idrees

Represents an educational board, university network, or school system.

Field	Type / Format	Constraints
name	string	Required, trimmed
board	string	Required — e.g. Federal, Punjab Board, HEC
type	enum	School, College, University, Institute
email	string	Required, lowercase
phone	string	Required
address	embedded object	street, city, province, postalCode, country
rating	number	0 to 5
image	string	Cover hero image URL
status	enum	Active, Pending, Inactive
adminId	ObjectId → User	The primary Institute Administrator
timestamps	Date (createdAt/updatedAt)	Automatic

Relationships:

- Has one lead User as adminId (Institute Admin).
- Has many Campus records (one-to-many).

Model 3: Campus — Owner: Idrees

A specific brick-and-mortar educational campus branch under an institute.

Field	Type / Format	Constraints
instituteId	ObjectId → Institute	Required, indexed
name	string	Required — e.g. "Main Canal Road Campus"
address	embedded object	street, city, province, postalCode, country
phone	string	Front desk / branch contact
email	string	Lowercase
managerId	ObjectId → User	The appointed Campus Manager
status	enum	Active, Pending, Inactive
timestamps	Date (createdAt/updatedAt)	Automatic

Relationships:

- Belongs to one Institute.
- Has one lead User as managerId (Campus Manager).
- Has many TeacherProfile, StudentProfile, ClassSchedule, ExamSchedule, and FeeRecord records.

Model 4: TeacherProfile — Owner: Habil

Stores academic and employment metadata for teachers beyond base login credentials.

Field	Type / Format	Constraints
userId	ObjectId → User	Required, unique, indexed
campusId	ObjectId → Campus	Required, indexed
department	string	e.g. "Computer Science", "Mathematics"
designation	string	e.g. "Professor / HOD", "Assistant Professor", "Lecturer"
qualification	string	e.g. "Ph.D. Computer Science", "M.Phil"
phone	string	Direct contact
subjects	array of strings	Courses authorized to teach
status	enum	Active, On Leave, Inactive
timestamps	Date (createdAt/updatedAt)	Automatic

Relationships:

- Belongs to one User.
- Belongs to one Campus.
- Has many ClassSchedule and TeacherAttendance records.

Model 5: StudentProfile — Owner: Habil

Stores enrollment, academic standing, and guardian details for students.

Field	Type / Format	Constraints
userId	ObjectId → User	Required, unique, indexed
campusId	ObjectId → Campus	Required, indexed
rollNo	string	Required, unique per campus
program	string	e.g. "BS Computer Science", "FSc Pre-Medical"
section	string	e.g. "Section A", "Section B"
semester	number	1 to 12
subjects	array of strings	Currently enrolled course titles
guardianName	string	Parent or legal guardian full name
guardianPhone	string	Emergency / parent telephone
phone	string	Student personal telephone
enrollmentStatus	enum	Active, Pending, Suspended, Graduated
cgpa	number	0.0 to 4.0
timestamps	Date (createdAt/updatedAt)	Automatic

Relationships:

- Belongs to one User.
- Belongs to one Campus.
- Has many StudentAttendance, FeeRecord, and Performance records.

Model 6: ClassSchedule — Owner: Habil

Weekly scheduled classroom lectures and lab sessions.

Field	Type / Format	Constraints
campusId	ObjectId → Campus	Required, indexed
teacherProfileId	ObjectId → TeacherProfile	Required
teacherName	string	Cached instructor name for fast tabular queries
subject	string	Required — course title
dayOfWeek	enum	Monday – Sunday
startTime	string	e.g. "08:30"
endTime	string	e.g. "10:00"
room	string	e.g. "Lab 3", "Room 204"
section	string	e.g. "Section A"
semester	number	Target semester
program	string	e.g. "BS Computer Science"

Field	Type / Format	Constraints
timestamps	Date (createdAt/updatedAt)	Automatic

Relationships:

- Belongs to one Campus.
- Belongs to one TeacherProfile.

Model 7: ExamSchedule — Owner: Habil

Formal examinations and evaluation datesheets.

Field	Type / Format	Constraints
campusId	ObjectId → Campus	Required, indexed
subject	string	Required — course title being evaluated
examType	enum	Quiz, Midterm, Final, Practical
date	date	Required — scheduled date of examination
time	string	e.g. "09:00 - 12:00"
room	string	e.g. "Exam Hall A"
invigilator	string	Name of assigned supervising proctor
totalMarks	number	e.g. 100
program	string	Enrolled degree program
section	string	Target class section
timestamps	Date (createdAt/updatedAt)	Automatic

Relationships:

- Belongs to one Campus.

Model 8: TeacherAttendance — Owner: Habil

Records daily check-in and attendance status for faculty.

Field	Type / Format	Constraints
campusId	ObjectId → Campus	Required, indexed
teacherProfileId	ObjectId → TeacherProfile	Required, indexed
date	date	Required, normalized to midnight
status	enum	Present, Absent, Late, On Leave
checkInTime	string	e.g. "08:15 AM"
timestamps	Date (createdAt/updatedAt)	Automatic

Relationships:

- Compound unique index on (teacherProfileId, date) — prevents duplicate entries per day.

Model 9: StudentAttendance — Owner: Habi

Records per-lecture or daily attendance for enrolled students.

Field	Type / Format	Constraints
campusId	ObjectId → Campus	Required, indexed
studentProfileId	ObjectId → StudentProfile	Required, indexed
subject	string	Required — subject/course name for the session
section	string	Class section
date	date	Required — date of lecture
status	enum	Present, Absent, Late, Excused
timestamps	Date (createdAt/updatedAt)	Automatic

Relationships:

- Compound unique index on (studentProfileId, subject, date).

Model 10: FeeRecord — Owner: Habi

Semester vouchers, tuition installments, and payment verification.

Field	Type / Format	Constraints
campusId	ObjectId → Campus	Required, indexed
studentProfileId	ObjectId → StudentProfile	Required, indexed
voucherNo	string	Required, unique — banking voucher tracking code
feeType	enum	Tuition Fee, Admission Fee, Semester Fee, Exam Fee, Transport Fee, Other
amount	number	Required — payable fee amount in PKR
month	string	Billing cycle / term — e.g. "Spring 2026"
dueDate	date	Required — payment deadline
paidDate	date	Optional — null if unpaid
status	enum	Pending, Paid, Overdue, Cancelled
paymentMethod	string	Bank Transfer, Cash, Online / Kuickpay, N/A
transactionRef	string	Banking receipt or reference number
timestamps	Date (createdAt/updatedAt)	Automatic

Relationships:

- Belongs to one Campus and one StudentProfile.

Model 11: Performance — Owner: Habi

Examination marks, grades, GPA calculations, and instructor remarks.

Field	Type / Format	Constraints
campusId	ObjectId → Campus	Required, indexed
studentProfileId	ObjectId → StudentProfile	Required, indexed
subject	string	Required — course title
marks	number	Required — numerical score attained
totalMarks	number	Default 100
grade	enum	A+, A, B+, B, C+, C, D, F
gpa	number	0.0 to 4.0 — term GPA
examType	enum	Midterm, Final, Assignment, Quiz
semester	number	Term number
remarks	string	Instructor notes or performance observations
timestamps	Date (createdAt/updatedAt)	Automatic

Relationships:

- Belongs to one Campus and one StudentProfile.

13. Entity Relationships

The diagram below shows how the eleven models connect; field-level detail for each entity is defined in Section 12.

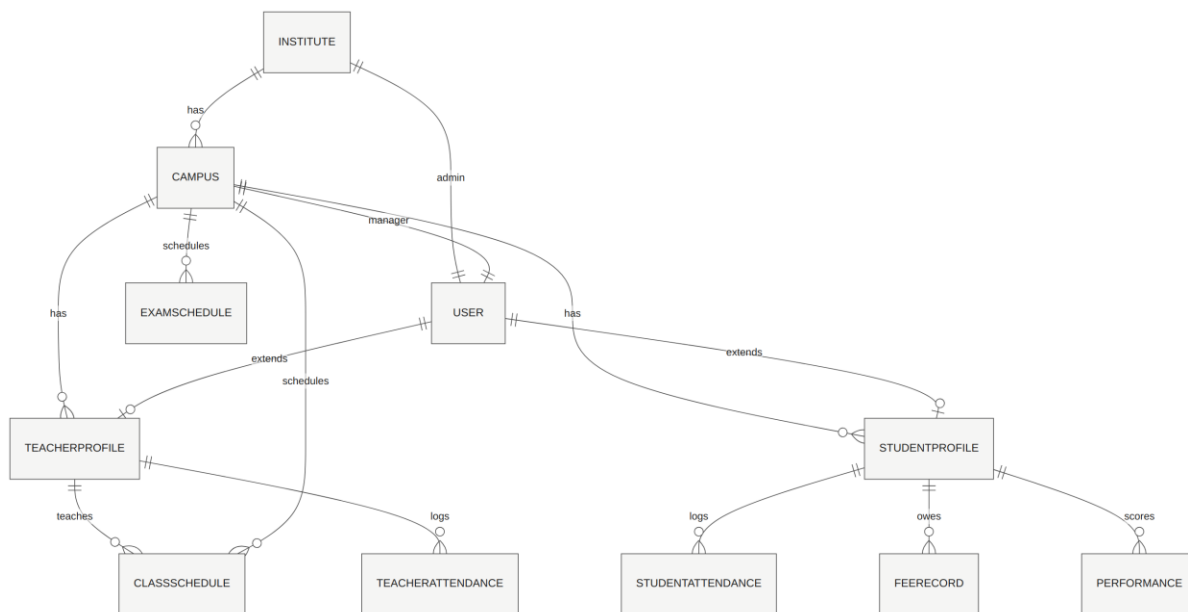


Figure 5 — Entity relationship diagram

14. Main Business Workflows

Workflow A — Onboarding a new Institute and its first Campus

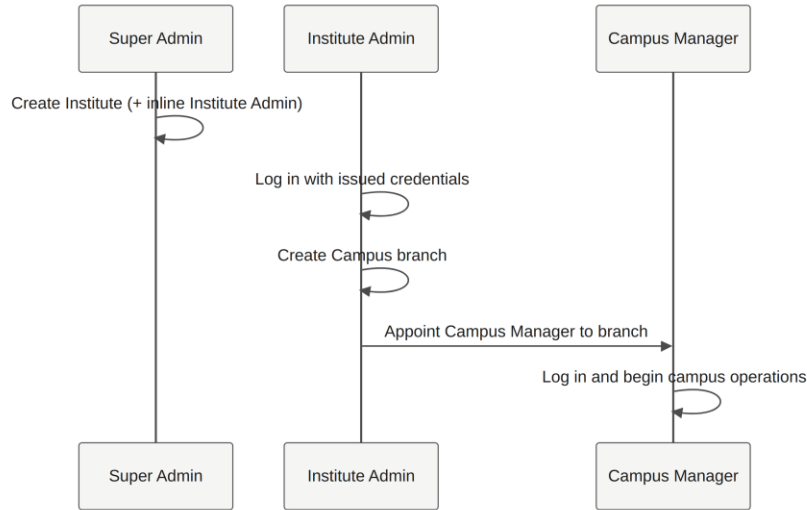


Figure 6 — Institute and campus onboarding

Workflow B — Student admission through to grading

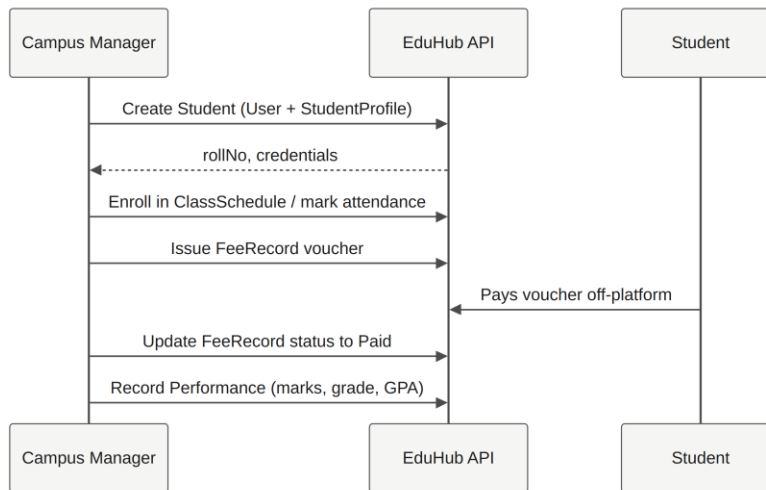


Figure 7 — Student lifecycle: admission to grading

Workflow C — Daily attendance cycle

The Campus Manager marks TeacherAttendance and StudentAttendance per day. The compound unique indexes on both collections prevent duplicate entries for the same person on the same date, and for students, the same subject.

15. Request / Response Flow

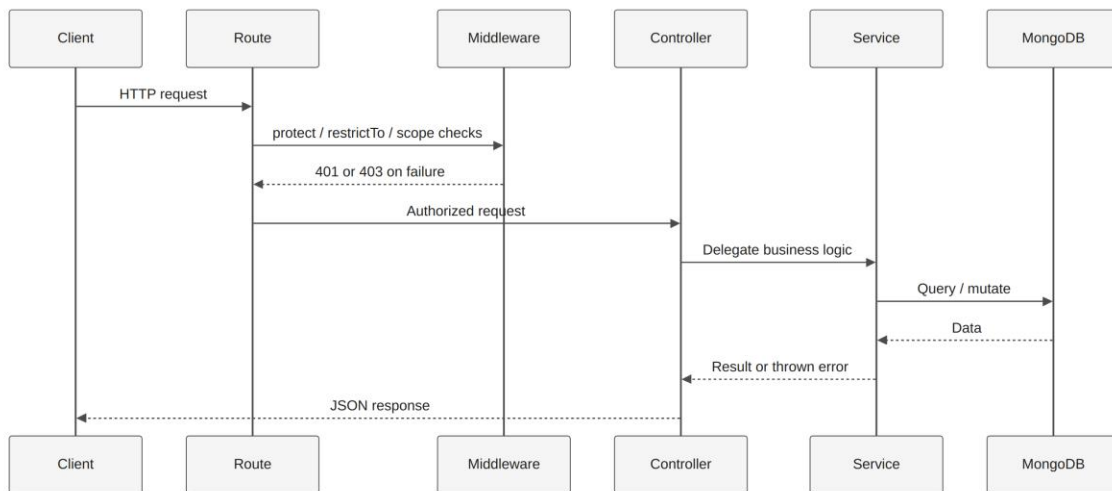


Figure 8 — Request and response flow through the layered backend

16. Error Handling

Note. This section reflects a proposed approach for a production-quality implementation; it is not detailed in the source requirements.

- A single centralized Express error-handling middleware catches all thrown or forwarded errors and formats a consistent JSON error envelope: { success: false, message, ...optionalDetails }.
- Services throw typed errors — for example `NotFoundError`, `ValidationError`, `AuthError` — rather than returning raw Mongoose errors, so the controller layer never has to interpret database-specific error shapes.
- Duplicate-key errors (duplicate email, rollNo, voucherNo, or attendance compound keys) are translated into a 409 Conflict with a field-specific message.
- Unhandled promise rejections in async route handlers are caught via an `asyncHandler` wrapper rather than repeated `try/catch` in every controller.

17. Validation

- Schema-level validation via Mongoose (required, enum, unique, min/max) as defined per model in Section 12 and the source requirements.
- Request-level validation, proposed: validate request bodies before they reach services, so malformed payloads never reach the database layer.
- Role and enum fields — `User.role`, `FeeRecord.status`, `Performance.grade` — are restricted to the exact enum values specified in the schema; the API rejects any other value at the model layer at minimum.

18. Security Considerations

Area	Approach
Password storage	Hashed with bcryptjs; passwordHash is excluded from default queries
Token security	JWT signed server-side; short-lived expiry recommended — proposed, not specified
Authorization	Enforced per-route via restrictTo and scope middleware, never left to the client
Tenant isolation	Every campus-level query is filtered by campusId; every institute-level query by instituteld
Input handling	Mongoose schema validation as a baseline; additional sanitization against injection is a proposed hardening step
Transport	HTTPS assumed in all deployment environments — proposed

Note. Rate limiting, CORS policy specifics, and request sanitization middleware (for example helmet or express-rate-limit) are not mentioned in the source requirements. They are recommended as standard hardening but are not treated as confirmed requirements here.

19. Logging and Monitoring

Note. This is a proposed approach; it is not specified in the source requirements.

- Request logging, for example morgan, in development for endpoint-level visibility during the two-track development process.
- Centralized error logging inside the error-handling middleware (Section 16) so failures are traceable without exposing stack traces to clients.
- No monitoring or alerting infrastructure is specified; console or log-file output is treated as the starting baseline.

20. Deployment Architecture

Note. The source requirements do not specify hosting. This reflects a standard setup consistent with the stated stack.

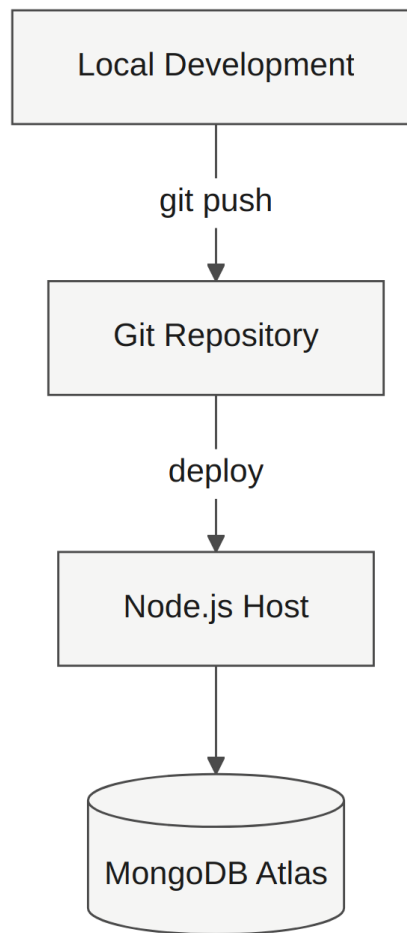


Figure 9 — Deployment overview

- Application — a Node.js process running the Express API.
- Database — MongoDB Atlas (managed) or a self-hosted MongoDB instance.
- Environment configuration (DB connection string, JWT secret, port) is supplied via environment variables and not committed to source control.

21. Development Workflow & Work Distribution

The project is split into two independently developed tracks so Idrees and Habil can work without merge conflicts.

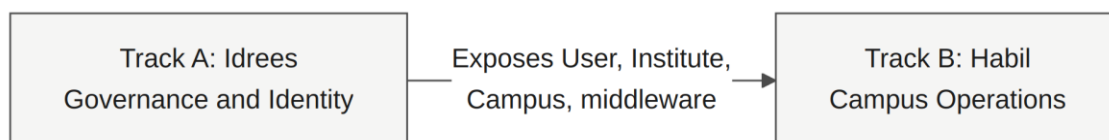


Figure 10 — Track ownership

Work Distribution

Person	Track	Models Owned	Responsibilities	Priority
Idrees	A — Core Governance & Multi-Tenancy	User, Institute, Campus	Auth (JWT, hashing); protect, restrictTo, instituteAdminScope, campusAdminScope middleware; Super Admin & Institute Admin services, controllers, routes	High — blocking
Habil	B — Campus Operations & Academic Execution	TeacherProfile, StudentProfile, ClassSchedule, ExamSchedule, TeacherAttendance, StudentAttendance, FeeRecord, Performance	Faculty / student CRUD including linked User creation; timetables, exams, attendance, fee vouchers, grading	High — starts in parallel using pre-seeded IDs

This split is intentionally balanced by ownership boundary rather than page count: Idrees owns fewer models but they carry the authentication and scoping logic every other endpoint depends on. Habil owns more models, but they are structurally similar — campus-scoped and CRUD-shaped — and do not require building the security layer from scratch.

Execution Plan — Idrees

Day	Task
1	User, Institute, Campus schemas with validators and compound indexes
2	JWT issuance, bcryptjs hashing, protect / restrictTo / instituteAdminScope middleware
3	superAdminService / superAdminController — institute creation with inline admin, admin listing
4	instituteAdminService / instituteAdminController — branch management, Campus Manager appointment
5	End-to-end test: token generation, Super Admin login, institute creation, new Institute Admin login

Execution Plan — Habil

Day	Task
1	8 operational schemas — TeacherProfile through Performance
2	Faculty and student CRUD in campusAdminService, including automatic linked User creation
3	Class timetable and exam datesheet services and endpoints
4	Faculty / student attendance and fee voucher issuance and status updates
5	Gradebook / performance logic — score-to-grade, GPA mapping; Postman test pass over all campus endpoints

Integration Protocol

- Contract-first — Habil depends only on User and Campus IDs, which Idrees seeds on Day 1, so Habil is not blocked and develops against pre-seeded test IDs.
- Branching — main is fed by feature/auth-and-governance (Idrees), merged first, then feature/campus-operations (Habil) is rebased onto main.
- Seeding — after merge, npm run seed populates all 11 collections with demo data.

Demo Credentials

For review and testing only — not production secrets.

Role	Email	Password
Super Admin	superadmin@eduhub.com	Admin@123
Institute Admin	instituteadmin@eduhub.com	Admin@123
Campus Manager	campusadmin@eduhub.com	Admin@123
Teacher	arshad.khan@pgc.edu.pk	Teacher@123
Student	hamza.ali@student.pgc.edu.pk	Student@123

22. Testing Strategy

Per the source requirements, endpoint verification is done manually with Postman, one module at a time, following each person's execution plan. Recommended minimum coverage:

Layer	Scenarios
Auth	Login success / failure, inactive account rejection, token expiry, self-registration
Super Admin	Institute CRUD, institute + inline admin creation, user activation toggle
Institute Admin	Branch CRUD scoped to own institute, rejection when targeting another institute's campus
Campus Manager	Faculty / student CRUD, duplicate rollNo / voucherNo rejection, attendance duplicate-entry rejection, fee status transitions, grade and GPA recording

Note. Automated unit or integration testing (for example Jest and Supertest) is not specified in the requirements, which name Postman explicitly. Automated tests are a reasonable future addition but are not presented here as an existing requirement.

23. Future Scalability Considerations

- Teacher / Student self-service endpoints — the current requirements only define Campus Manager-side CRUD over teacher and student data; role-scoped read and limited write endpoints for teachers and students viewing their own schedule, attendance, and grades would be a natural next step.
- Notifications — fee due-date reminders, attendance alerts, and grade publication notices are not in scope today but are common extensions for this domain.
- Payment gateway integration — FeeRecord.paymentMethod includes Online / Kuickpay, suggesting future integration with a live payment processor rather than manual status updates.
- Reporting and exports — transcript or attendance report generation (PDF or CSV) is a likely future requirement given the gradebook and attendance data already modeled.
- Horizontal scaling — as institute and campus counts grow, indexes on instituteld and campusId, already specified, support sharding by tenant if needed.

24. Conclusion

EduHub's backend is structured around a clear three-tier tenancy model, a conventional layered architecture — routes, middleware, controllers, services, models — and a work split that lets both developers build independently against a shared, contract-first foundation. This documentation reflects only what is defined in the source requirements, with proposed hardening in Sections 16 and 18 through 20, and Section 22, explicitly labeled so it is not mistaken for a stated requirement during review.